Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

CS3081 Cloud Computing — Mini Project Report

Name: Chandra Prakash · Roll number: 230101027 · Repository: https://github.com/cp18112004/cloud-computing-assignment (tag `final`)

Conventions. Measured values come from files in `results/` produced by the GitHub Actions runs listed in the README (each figure cites its file). Statements marked *Assumption* are not measured. All runs used GitHub-hosted `ubuntu-24.04` runners (4 vCPU, 16 GB RAM) in place of a laptop; the cluster is the same `cluster/k3d.yaml` either way.

```
GitHub: Actions (ci, teacher, train, service, deploy, report, package) | Releases model-v1/-v2/-base
   | make up (scripts/up.sh)
k3d cluster "cmdparser" (one k3s node)   :8080 -> Traefik Ingress   :8333 -> SeaweedFS S3
 |- ns data:     SeaweedFS (PVC 10Gi): buckets datasets/, models/v1/, models/v2/ <- storage-seed Job
 |- ns pipeline: Ollama phi3.5 <- teacher-worker Jobs (SHARD_INDEX/COUNT) -> datasets/teacher/shard-i
 |               clean-dataset Job -> datasets/v1/train.jsonl + train.sha256
 '- ns app:      Deployment cmdparser x2 (FastAPI, Pythia-70M from s3://models/v1/)
                 <- Service <- Ingress /parse ;  HorizontalPodAutoscaler 1..2 replicas at 70 % CPU
```

1 Task 1 — Object storage and parallel teacher labelling

Architecture. `cluster/k3d.yaml` defines a one-node k3s v1.31.4 cluster with host ports 8080 → Traefik and 8333 → S3. Namespace `data` runs SeaweedFS 3.80 (master, volume, filer and S3 gateway in one Pod on a 10 Gi local-path PVC). Its S3 identity file and the clients' `AWS_ACCESS_KEY_ID/SECRET` are Kubernetes Secrets generated with random values by `scripts/secrets.sh` (kept only in the git-ignored `.secrets/`). The `storage-seed` Job creates the buckets `datasets` and `models` and uploads `commands_pool.jsonl`. Namespace `pipeline` runs the teacher, Ollama 0.9.6 serving `phi3.5` (3.8B, Q4_0, pulled by an init container) with `OLLAMA_NUM_PARALLEL` = number of workers, and one Job per worker rendered from `k8s/pipeline/worker-job.yaml`.

Prompt and settings. `prompts/teacher_prompt.txt` gives the task, the exact four keys, the eight intents with one-line meanings, the three slot rules and 14 examples written by us (two from the handout, none from the validation set or the pool), ending in `Command: {command}\nJSON:`. Options: temperature 0, seed 42, `num_ctx` 2048, `num_predict` 128, and a stop at the first newline (plus phi3.5's own stop tokens). The first validation run reached only 90 % JSON validity; 9 of the 10 failures were a correct JSON line followed by `\n\n(Note: …)` (`results/task1/first-validation-run/`). With the newline stop the teacher reached 100 %. Replies are stored exactly as Ollama returns them.

Sharding. Worker *i* takes the lines whose 0-based number *n* satisfies *n* mod SHARD_COUNT = SHARD_INDEX. Shards are therefore disjoint, cover the whole file, differ in size by at most one record and contain a similar mix of commands, so the workers finish together. Each worker writes only its own object `datasets/teacher/shard-<i>.jsonl`, so no locking is needed. It re-uploads the object every 25 records, and a restarted Pod resumes after the records already there (checked against the expected id sequence). The teacher is the bottleneck (~4 tokens/s per stream on a 4-vCPU CPU). We therefore ran the same Job template on **6 clusters (one per GitHub runner) × 2 concurrent workers = SHARD_COUNT 12**. Each cluster's `workers_running_pool-r*.txt` shows its two worker Pods Running with different SHARD_INDEX values. The 12 objects were then loaded into one SeaweedFS and cleaned there by the `clean-dataset` Job.

Results (measured). Each worker processed 250 commands in 1,907–2,510 s (`results/task1/worker_pool-r*_shard-*.log`). Wall time for the 3,000 commands was ≈ 49 min, against ≈ 3.4 h estimated for one runner (*assumption*: the per-cluster rate scaled to 3,000 commands).

Input	(a) not JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	Duplicates	Kept
3,000	4	42	0	152	45	**2,757**

4 + 42 + 0 + 152 + 45 + 2,757 = 3,000 (`results/task1/clean_report.json`). The coverage check found 3,000 ids in 12 shards, with 0 missing and 0 repeated. Typical rejections:

- a. two JSON objects for a two-step command (C0075);
- b. a missing `place` key (C0045);
- d. an altered value, e.g. `"date": "this day"` for *is this the hottest day in mangalore* (C0020).

`train.sha256` = `f536dfca…f37b64d0`, and re-running `worker/clean.py` locally on the committed shards gives the same digest (`results/task1/clean_checksum_check.txt`). The first 1,500 kept records end at C1618.

Limitations and improvements. Decoding a 3.8B model on CPU is slow. The multi-runner fan-out uses separate clusters, so the final store receives the shards by seeding rather than directly from the workers. Rule (d) is a substring test, so a well-formed but wrong value that happens to occur in the command passes. *Improvements:* one multi-node cluster or a GPU node with a single store; schema-constrained decoding for the teacher; semantic slot checks in `clean.py`.

2 Task 2 — Training the student (sequence-level distillation)

Setup. `training/config.yaml` records seed 42, learning rate $3{\cdot}10^{-4}$, batch size 16, 8 epochs, max sequence length 96 and the library versions (torch 2.5.1+cpu, transformers 4.46.3, tokenizers 0.20.3, safetensors 0.4.5, numpy 2.1.3, Python 3.12). The base model `EleutherAI/pythia-70m` is pinned to commit `a39f36b`. `make train` reads only that file; `make train SUBSET=50` ran in 50 s (`results/task2/train_subset50.log`). Each example is `Command: <text>\nJSON: <teacher JSON><eos>`. The loss covers the JSON reply and eos only, because the prompt tokens are masked. AdamW is used with linear warm-up (5 %) and decay. Training uses the first 1,500 cleaned records (C0001–C1618, 39 tokens on average, none truncated). v1 took 1,361 s on the CPU. v2 is the same run with **one more epoch** (9 epochs, 1,530 s).

Epoch	1	2	3	4	5	6	7	8	9
v1 loss (`training/loss.csv`)	0.587	0.380	0.155	0.101	0.071	0.041	0.022	0.008	–
v2 loss (`training/loss_v2.csv`)	0.572	0.363	0.174	0.115	0.070	0.050	0.029	0.016	0.005

Artefacts. The GitHub Releases `model-v1` and `model-v2` hold `model.safetensors`, the tokenizer files, `config.json`, `version.json` (version information), `model.sha256` and a deterministic tarball. `models/v1/model.sha256` lists the SHA-256 of every file and of the tarball; CI checks the downloaded Release with `sha256sum -c`. The storage-seed Job copied both versions from the Releases into `s3://models/v1/` and `s3://models/v2/`, verifying every file (`results/task2/models_to_object_storage.log`, `models_bucket.txt`).

Evaluation on validation.jsonl (measured, `score.py`). Raw outputs are in `results/{teacher,pythia_base,student_v1,student_v2}_validation_predictions.jsonl`, with the reports beside them (`results/teacher_validation.json` for the teacher).

Model	JSON valid	Intent	Date	Time	Place	Full frame
Teacher phi3.5 (3.8B, Ollama)	100.0	89.0	78.0	88.0	91.0	**58.0**
Pythia-70M, untrained	0.0	0.0	0.0	0.0	0.0	**0.0**
Student v1 (served model)	100.0	87.0	81.0	87.0	84.0	**50.0**
Student v2 (+1 epoch)	99.0	84.0	85.0	89.0	80.0	**51.0**

What changed. The untrained model never produces the format: after `JSON:` its greedy continuation is a newline, so every output is empty. After distillation, the 70M student produces valid four-key JSON every time. It reaches 86 % of the teacher's full-frame accuracy (50 vs 58), and is even slightly better on dates. On 58 of 100 commands its output is identical to the teacher's: it learnt the teacher's conventions, including its mistakes. v2's extra epoch lowered the training loss but did not help overall (one invalid output, lower intent accuracy). This is a sign of over-fitting to 1,500 examples, so v1 stays the served version.

Sequence-level vs logit-based distillation. (a) *Sequence-level KD* (Kim & Rush, 2016) trains the student on the teacher's **generated output sequences**. Here the target is the teacher's one-line JSON reply for each command, and the loss is ordinary next-token cross-entropy on those hard labels. (b) *Logit-based (word-level) KD* trains the student to match the teacher's **probability distribution over the vocabulary at every token position** (soft targets, typically a KL divergence on temperature-softened logits). The student thereby also learns which alternatives the teacher considered plausible. (c) It is **not possible here** for two reasons. First, the two models use different tokenizers and vocabularies: Phi-3.5 has a 32,064-token Llama-style vocabulary, Pythia a 50,304-token GPT-NeoX BPE. So there is no token-by-token correspondence between their distributions. Second, the teacher is only reachable through Ollama's generate API, which returns text, not full-vocabulary logits. (d) **A limitation of our approach:** the student can only copy the teacher's single greedy answer. It inherits the teacher's systematic errors (for example slot boundaries that differ from the SLURP labels), it gets no signal about the teacher's

uncertainty, and cleaning removes only *malformed* replies, not wrong but well-formed ones. That last point caps the student near the teacher's 58 % full frame.

3 Task 3 — The inference service in a container

Implementation. `service/app.py` is a FastAPI app on uvicorn, listening on port 8000.

- *Start-up.* A background thread lists `MODEL_URI` (e.g. `s3://models/v1/`), downloads every object from `S3_ENDPOINT` with boto3 and checks each file against `model.sha256`. It then loads the model (fp32, with the thread count taken from the cgroup CPU quota) and runs one warm-up generation. Only after that does `/readyz` return 200; before, it returns 503. `/healthz` is always 200. Credentials come only from `AWS_ACCESS_KEY_ID`/`SECRET`.
- */parse.* Greedy decoding, shared with `training/predict.py` so offline scores and the service agree, stops at eos, at a newline or at `MAX_NEW_TOKENS`. With `FAULT_INJECT=1` the output is replaced by `not json` before validation.
- *Validation.* Every output is checked with jsonschema against `schema.json`. A valid output returns 200 with exactly the four-key object; anything else returns 500 `{"error": "invalid_model_output"}`. Both responses set `X-Model-Version` and `X-Served-By` (the Pod's hostname).
- *Image.* `service/Dockerfile` uses python:3.12.8-slim with the CPU-only torch wheel. All 45 packages are pinned, installed with `--no-deps` and then checked with `pip check`. The image runs as a non-root user and contains no weights.

Results (measured, `results/task3/`).

- One command (`make image`) builds the image in 50 s; it is **0.948 GB** (limit 2 GB).
- No `*.safetensors`, `*.bin`, `*.pt` or `*.gguf` file exists in the image (`image_no_weights.txt`).
- Started with `--cpus 1 --memory 1g --memory-swap 1g`: `/healthz` returned 200 after 1.6 s, `/readyz` returned 503 until 9.3 s and then 200 (`startup_probes.txt`).
- 50 sequential `/parse` requests: **50 × HTTP 200**, median 404 ms, p95 461 ms, max 537 ms (`sequential_50.txt`).
- **Peak memory 682,262,528 B = 651 MiB** (cgroup `memory.peak`, which includes the page cache of the downloaded weights). The highest `docker stats` sample was 374.6 MiB. The container recorded OOMKilled=false and 0 `oom_kill` events (`peak_memory.txt`).
- With `FAULT_INJECT=1`: `HTTP/1.1 500` with body `{"error":"invalid_model_output"}` (`fault_inject.txt`).

Limitations and improvements. Each Pod downloads 282 MB at start-up, and fp32 weights double the memory of fp16. *Improvements:* fp16/int8 weights, a node-local model cache, and batching requests inside the service.

4 Task 4 — Kubernetes deployment, scaling and reliability

Manifests (`k8s/app/`).

- *Deployment:* 2 replicas, RollingUpdate with maxSurge 1 / maxUnavailable 0. Requests 500m / 512Mi, limits 1 CPU / 1 GiB. Readiness probe on `/readyz` every 2 s, liveness probe on `/healthz`. A preStop sleep of 8 s and a 40 s grace period. An emptyDir at `/tmp` holds the downloaded model, and credentials come from the Secret `s3-credentials`.
- *Service and Ingress:* a ClusterIP Service and a Traefik Ingress with an Exact `/parse` rule.
- *HPA:* autoscaling/v2, min 1, max 2, 70 % CPU, with a 60 s scale-down stabilisation window.

`scripts/k8s_experiments.sh` runs every experiment under load, using the standard Locust file through the Ingress (`deploy.yml`). For the controlled experiments (cold start, load balancing, rolling update, recovery) the HPA minimum was pinned to 2 so that exactly two replicas existed. It was restored to 1 for the autoscaling test.

Results (measured, `results/task4/`, run on a fresh GitHub runner after `make up`).

- **Ingress.** `curl -i -X POST localhost:8080/parse` returns 200 with the four-key JSON, `X-Model-Version: v1` and `X-Served-By: cmdparser-…` (`ingress_parse.txt`). The applied requests, limits and probes are in `app_resources.txt`.
- **No ready endpoints while loading.** After all Pods were deleted (cold-start run 1), the new Pods were Running but not Ready for ≈ 12 s. During that window the Service showed `ready_endpoints=0 not_ready_endpoints=2`, and the Ingress answered 404 instead of routing to a loading Pod (`coldstart_run1_watch.txt`). The Pod logs show `downloaded s3://models/v1/model.safetensors from http://seaweedfs.data.svc.cluster.local:8333 (281715176 bytes)` followed by `model v1 loaded and ready after 10.0s` (`coldstart_run1_*.log`).

Experiment (under the standard 4-user load)	Requests	Failed	Outcome
Cold start ×3: delete all Pods, creation → Ready	–	–	run 1: 15 / 13 s, run 2: 13 / 13 s, run 3: 14 / 14 s → **median 14 s**
Load balancing, exactly 2 Ready replicas (standard command)	557	0	**280 vs 281** successful = 49.9 % / 50.1 % per replica
Rolling update v1 → v2 (`kubectl set env`), then `kubectl rollout undo`	1,790	4 (**0.22 %**)	after the rollback, 6/6 responses carry `X-Model-Version: v1`
Recovery: delete one Pod	752	0 (**0 %**)	1/2 Ready at 11:50:35, **2/2 Ready again at 11:50:48** (13 s)
HPA (min 1, max 2, 70 %): idle → load → idle	1,356	0	2→1 at 11:54:37 (0 % CPU); 129 % → **2 Ready at 11:55:29**; load off 11:59:56 → **1 at 12:01:37**

The four failures in the rolling-update run are not connection errors. All four are HTTP 500 `invalid_model_output` from v2 on the one validation command it gets wrong (V026: v2 omits two keys; it is 99 % valid). v2 served 379 requests, ≈ 3.8 passes over the 100 commands (`rolling_headers.json`, `rolling_failures.csv`). The rollout itself (maxSurge 1, maxUnavailable 0, readiness gate and preStop sleep) caused no errors. Timelines: `rolling_timeline.txt`, `recovery_replicas.txt`, `hpa_timeline.txt`, `hpa_describe.txt`.

Limitation. The whole cluster is one k3s node on one machine (a GitHub runner here, a laptop otherwise). Replicas, the HPA and recovery are demonstrated only *within* one node: a machine, Docker or node failure takes everything down, so this setup does not show fault tolerance across machines. SeaweedFS also runs as a single replica. metrics-server samples only every ~15 s, so the HPA reacts with a delay. *Improvements:* a multi-node cluster with Pod anti-affinity and a PodDisruptionBudget, replicated object storage, and a startupProbe.

5 Task 5 — Subnetting and network access control

Allocation from 10.20.0.0, largest first, using the smallest prefix with 2^(32−p) − 2 ≥ hosts (working in `network/subnets.md`; recomputed by `tests/test_network.py`).

Subnet	Hosts needed	Network	Prefix	Usable host range	Broadcast	Usable hosts
app	1,000	10.20.0.0	/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0	/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

NACL for the public subnet (`network/nacl.json`).

- *Inbound:* 100 deny all from 198.51.100.0/24; 110 allow tcp 80 from 0.0.0.0/0; 120 allow tcp 443 from 0.0.0.0/0; 130 allow tcp 22 from 10.20.5.128/27; 140 allow tcp 1024–65535 from 10.20.0.0/22 (replies from app); 32766 deny all.
- *Outbound:* 100 allow tcp 8000 to 10.20.0.0/22; 110 allow tcp 1024–65535 to 0.0.0.0/0 (replies to clients); 32766 deny all.

Measured: `nacl_sim.py --check` scores 6/6 on the sample packets (`results/task5/nacl_sample_check.txt`). It also scores 24/24 on our own edge cases (`network/packets_extra.jsonl`): the /27 boundaries, ports 1023/1024, UDP, and the abusive range on ports 22/80/443.

Why replies need their own rules. A NACL is stateless: it does not remember that a packet belongs to a connection it already allowed, so every packet, including a reply, is evaluated on its own. A client's HTTPS request is allowed by inbound rule 120, but the response travels to the client's ephemeral port and needs outbound rule 110. Likewise the app's replies reach the load balancer's ephemeral ports and need inbound rule 140.

Why order matters for 198.51.100.0/24. Rules are checked from the lowest number and the first match wins. A packet from 198.51.100.9 to port 443 matches both the deny (100) and the allow for 443 (120). The deny takes effect only because its number is lower than the allow it must override; numbered after the allows, it would never be reached for web traffic.

Limitation: the outbound ephemeral rule must allow 1024–65535 to anywhere, so the NACL alone cannot stop a new connection to a high port such as 5432 in the data subnet (our packet X24). Stateful security groups per tier would close that gap.

6 Task 6 — Reproducibility, testing and CI/CD

Reproducible build and deploy. From a fresh clone it takes three commands (`git clone`, `cd`, `make up`). `make up` on a fresh runner finished in **175 s** (`results/task6/make_up.log`): cluster, Secrets, both images, SeaweedFS, the seeding of datasets and models from the Releases, and the service behind the Ingress. `make down` deletes the cluster (`results/task6/make_down.log`).

CI (`.github/workflows/ci.yml`). The stages run in this order:

1. lint: ruff, YAML/JSON and shell syntax, plus a **gitleaks** scan of the full history (*no leaks found*, `results/task6/gitleaks.txt`);
2. unit tests: 20 pytest cases covering the cleaning rules and count identity, sharding without gaps or repeats, the service API with a fake model (headers, 503 before ready, `FAULT_INJECT`, invalid outputs never returned as 200) and the subnet/NACL checks;
3. evaluation: `gh release download`, then `sha256sum -c`, `training/predict.py` and `score.py` on validation.jsonl;
4. quality gate: `score.py --min-validity 95`.

With `model-v1` all four stages pass: validity 100, full frame 50 (https://github.com/cp18112004 /cloud-computing-assignment/actions/runs/42475512196). With the deliberately bad `model-base` (untrained Pythia-70M), stages 1–3 pass and **the quality gate fails** with `QUALITY GATE FAILED: json_validity 0.0 < 95.0` (https://github.com/cp18112004 /cloud-computing-assignment/actions/runs/42475510183, `results/task6/quality_gate_failed_run.txt`). `AI_USE.md` lists the AI tools used and how their output was checked.

Limitation: the evaluation stage uses the 100 validation examples only, so a regression on the hidden distribution could still pass. Downloads from GitHub occasionally returned HTTP 504, so the tool installs now retry and fall back to container images.

7 Task 7 — Performance, cost and review

Standard load test (measured). Command: `locust -f tools/locustfile.py --headless --users 4 --spawn-rate 4 --run-time 2m --host http://localhost:8080 --csv results/loadtest`, with exactly two Ready replicas behind the Ingress (`results/loadtest_stats.csv`, `results/task7/locust_standard_run.log`).

p50 latency	p95 latency	Throughput	Error rate	CPU per Pod (`kubectl top pods`)	Memory per Pod	Cold start
850 ms	**1,000 ms**	**4.68 req/s**	**0 / 557 = 0.0 %**	1,000m (= limit), 12 samples	median 378–435 MiB, max 442 MiB	**14 s** (median of 3)

Both Pods ran at their 1-CPU limit for the whole test (`results/task7/top_pods.txt`), so throughput is CPU-bound: ≈ 2 × 1 CPU / 0.42 s per request (the single-request latency in Task 3).

Cost (`data/rate_card.json`; each replica billed for its limits, `results/task7/cost.json`):

1. Per replica per hour = 1 vCPU × ₹3.50 + 1 GiB × ₹0.45 = **₹3.95**.
2. Two replicas = 2 × 3.95 = **₹7.90 per hour**.
3. Requests per hour = 4.681 req/s × 3,600 = **16,851.6**.
4. Cost per 1,000 requests = 7.90 / 16,851.6 × 1,000 = **₹0.469**.

Assumption: the rate measured at 4 users is sustained for the whole hour. At lower traffic, idle replicas still cost ₹3.95 per hour each.

SLO. 99 % of `POST /parse` requests return HTTP 200 within 30 s; the window is one standard load-test run. The error budget is 1 % × 557 = **5.57 requests**, and **0 were used (0 %)**, so the SLO was met (`results/task7/slo.json`). For comparison, the rolling-update run used 4 of its 17.9-request budget (22 %).

Component or concern	Evidence (a measured number)	Limitation found	Improvement proposed
Data and storage	3,000 → 2,757 records kept; `train.sha256` reproduced byte for byte; 12 shard objects of ≈ 43 KB	One SeaweedFS Pod, no replication or backup; one S3 identity shared by all namespaces	Replicated SeaweedFS (volume replication), versioned buckets, a scoped read-only identity per namespace
Model quality	Student v1 full frame 50 % vs teacher 58 %; JSON validity 100 %	Copies the teacher's slot mistakes; v2 (+1 epoch) did not improve	Better teacher prompt or voting, more data, schema-constrained decoding, larger student
Latency and throughput	p50 850 ms, p95 1,000 ms, 4.68 req/s with 2 Pods	One generation at a time per Pod; CPU-bound greedy decoding	int8/ONNX runtime, request batching, higher HPA maximum
Resource utilisation	1,000m CPU (at the limit), ≤ 442 MiB of a 1 GiB limit; Task 3 peak 651 MiB	Memory limit oversized; HPA reacts after ≈ 30 s	Right-size memory to 768 Mi; fp16 weights; shorter metrics interval
Reliability and recovery	Recovery 0 failures, 13 s to 2/2; rolling update 0.22 %; cold start 14 s	Single node; each Pod re-downloads 282 MB; v2's invalid outputs reached users	Multi-node with a PodDisruptionBudget; node-local model cache; canary release with automatic rollback on the 500 rate
Security and cost	gitleaks: no leaks; credentials only from Secrets; ₹0.469 per 1,000 requests	Replicas billed for limits even when idle (≥ ₹3.95/h); S3 identity has Admin rights	Scale to zero off-peak, lower requests; least-privilege S3 identity; NetworkPolicies

References

1. Kim, Y. and Rush, A. M. *Sequence-Level Knowledge Distillation.* EMNLP 2016.
2. Hinton, G., Vinyals, O. and Dean, J. *Distilling the Knowledge in a Neural Network.* 2015.
3. Bastianelli, E. et al. *SLURP: A Spoken Language Understanding Resource Package.* EMNLP 2020.
4. Biderman, S. et al. *Pythia: A Suite for Analyzing Large Language Models.* ICML 2023 (EleutherAI/pythia-70m model card).
5. Microsoft. *Phi-3.5-mini-instruct* model card; Ollama library `phi3.5`.
6. Documentation: k3d, k3s networking, SeaweedFS, Kubernetes (Deployments, Services, probes, HPA, Secrets, rolling updates, resource metrics pipeline), FastAPI, Docker, GitHub Actions, gitleaks, Locust, AWS VPC network ACLs, Google SRE book (SLOs).